



STRADIBOT REPORT

CS225A - Experimental Robotics
Stanford University

Joana MIZRAHI
Loïc POISSON
Harold GARDON
Anna MASSET



Stanford
University



The Stanford
Robotics
Center

CONTENTS

1 - Introduction	1
1.1 Project description and objectives	1
2 - Control and simulation	1
2.1 Finite State Machine	1
2.2 Results in the simulation	2
3 - Hardware and interaction	3
3.1 Robot description	3
3.2 Human-robot interaction	4
4 - Project Evaluation	4
4.1 Challenges and future work	4
Demo Videos and Repositories	5
5 - Demo Videos and Repositories	5
Appendices	

1 - INTRODUCTION

1.1 PROJECT DESCRIPTION AND OBJECTIVES

Previous violin-playing robots typically rely on rigid, hard-coded trajectories that struggle to adapt to the instrument's physical feedback. To address this, our project leverages Operational Space Control (OSC) to introduce compliance and robustness. Generating a good sound on the violin requires maintaining a specific downward force coupled with bow velocity. Working in the operational space allows us to decouple these tasks, enabling simultaneous closed-loop force control normal to the strings and oscillatory position control along the bowing path. Beyond continuous bowing, we also manage string transitions (detailed in the state machine section). Our primary objective is to validate this OSC controller in simulation before safely deploying it on physical hardware. Finally, the system integrates an Arduino to actuate string presses and a human-robot interface for commanding specific notes and continuous sequences.

2 - CONTROL AND SIMULATION

2.1 FINITE STATE MACHINE

Controller state machine

Before explaining the Finite State Machine (FSM), it is important to understand some of the key states that it comprises.

BOWING: We define a local frame on the target string (Figure 1): X lies along the string, Y is the bowing path, and Z is the string normal. Applying closed-loop force control along the Z-axis dynamically modulates sound and compliantly adapts to violin movement, ensuring robust performance and hardware safety.

CALIBRATING: Prior to bowing, each string's local frame is defined by manually placing the bow on the string. We record the end-effector's orientation; the columns of the resulting rotation matrix yield the X, Y, and Z directional vectors, which are then saved to our string database.

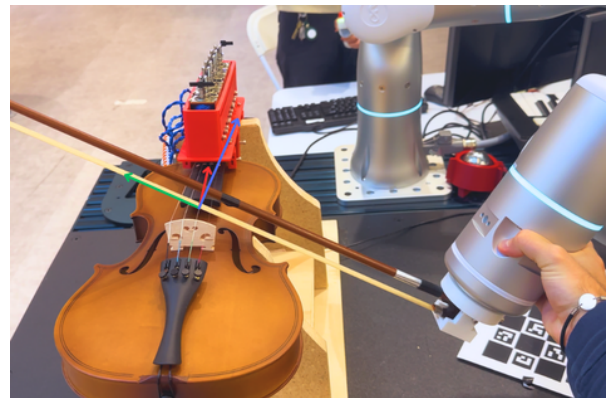
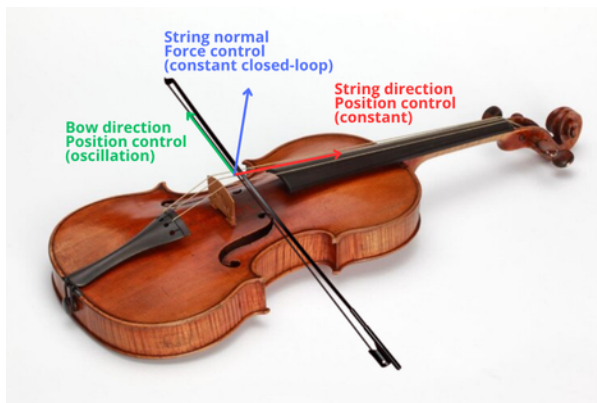


Figure 1: Control frames

The FSM of the controller can be found in the appendices. Note that this FSM has three different ‘modes’ but all of them are pretty similar so they have been put together in the schematics. The following table shows the gains and control types used for the different states.

State(s)	Control Type	K_P	K_V	K_I
Zeroing, Initializing	Joint Position	50	14	0
Homing, Placing, Contacting, Lifting, Moving	OSC (Position)	100	20	0
Bowing (Position Task)	OSC (Position)	100	20	0
Bowing (Force Task)	OSC (Force)	0.7	10	1.3

Link with the Arduino controller

To extend the system’s musical range, we developed a custom fretting module featuring 6 solenoids mounted on a 3D-printed fingerboard clamp. Each solenoid presses all four strings simultaneously, since the robotic arm only bows the target string it does not impact the goal key. The solenoids are driven by a 13 V supply via Arduino-controlled MOSFETs, generating up to 42 N of force to ensure robust string depression. The Python FSM commands the Arduino over a USB serial link using single-byte messages (0 to release, 1–6 to activate specific frets). To prevent premature actuation during string transitions, the FSM applies a non-blocking 0.2 s delay (FRET_SWITCH_DELAY), ensuring the fret engages only when the bow makes contact.

2.2 RESULTS IN THE SIMULATION

The full control pipeline was first validated in an OpenSai simulation (physics at 2000 Hz, FSM at 100 Hz) using a modeled Flexiv Rizon 4S and fake static violin. This environment was crucial for safely verifying core behaviors before switching to the real robot. Specifically, we validated the hybrid force-position controller’s ability to maintain a stable contact force (>1.2 N threshold) during continuous ± 20 cm bow strokes. We also verified the FSM’s reliability during string transitions, confirming that adjacent (direct lateral) and non-adjacent (2 cm lift) changes execute smoothly within 2–4 s. Finally, the simulation proved that the autonomous MIDI logic correctly parses .mid files to sequence bowing and solenoid commands across all strings, tested successfully on multi-string pieces like *Canon in C*. The transition between *BOWING* and *LIFTING* can be seen in Figure 2.

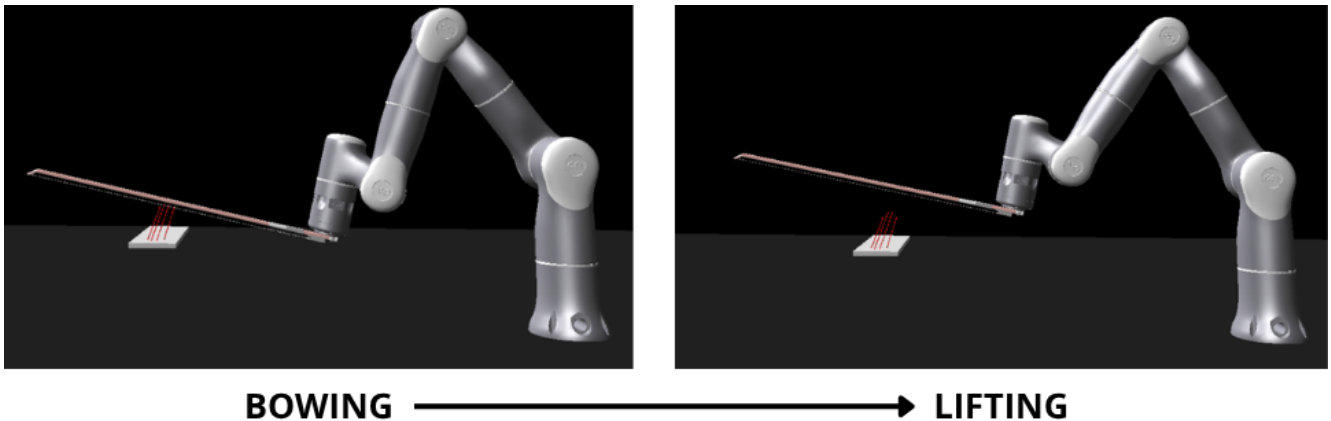


Figure 2: Simulation: bowing on string 3 then lifting to change string

3 - HARDWARE AND INTERACTION

3.1 ROBOT DESCRIPTION

The robotic arm

For this project, we use a Flexiv Rizon 4S arm. This robotic arm is useful to use because it has an integrated force sensor in the end-effector. This force sensor is essential for us since we plan on using closed-loop force control.

The first thing we need to do is to rigidly attach the bow to the end-effector of the arm. In order to do so, we use a 3D printed model, see Figure 3. This bow-holder is made to be attached directly onto the end-effector, it has an edge against which the bow is lying. Three screws then allow to hold the bow tightly so that it does not move. Finally, it is designed so that we still have access to the bow's screw that tightens the hair.

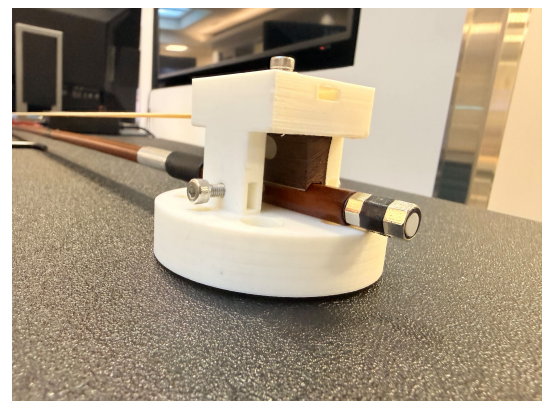
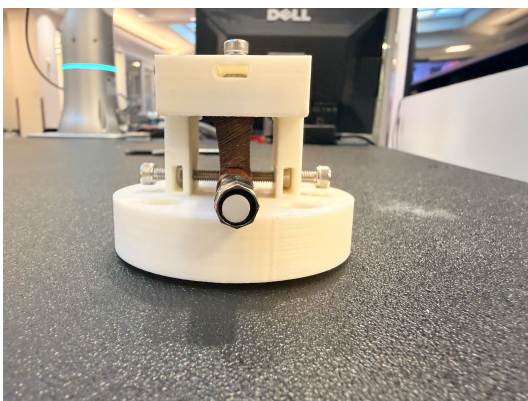


Figure 3: 3D printed piece to hold the bow.

The string presser



Figure 4: Left: Overview of the global setup. Right: close-up of the solenoids.

The second design we had to do for this project is the string presser. As explained before, if we want to be able to play a song with the violin, we need to be able to change the tonality of the different strings in real time. To do so, we built an Arduino Hardware connected to the state machine.

To hold these solenoids over the fingerboard, we had to design a final piece. Since we are asking for 40N, we need this piece to be tightly attached to the fingerboard. Thanks to this design, we are able to calibrate the solenoids by choosing where we place them on the fingerboard. This allows us to play a bigger range of keys.

Finally, to ensure a good sound with the bowing, it is very important to have a stable violin. Furthermore, we need to have the violin a little bit higher so that the arm can bow on each string. This is why we also built a stand to hold the violin, it can be seen in Figure 4.

3.2 HUMAN-ROBOT INTERACTION

The robot is able to play in two different regimes. The first one is an autonomous regime. Given a MIDI file (equivalent of a music sheet in computer language), the robot is able to play the full song autonomously. The way this works will be developed in the state machine.

The second regime is a human-robot interaction. With a plugged MIDI keyboard, we can play any key on the keyboard and the robot will play the same key on the violin. This would allow someone to play a piece of music.

4 - PROJECT EVALUATION

4.1 CHALLENGES AND FUTURE WORK

Several challenges arose during the development of StradiBot that informed both the final design and directions for future improvement.

- **CHALLENGES**

- **Force control:** Initial open-loop force and moment attempts yielded inconsistent acoustics. We ultimately achieved stable closed-loop force control because the natural compliance of the bow hair successfully absorbed control-induced vibrations.
- **String fretting:** Complex dynamic designs (e.g., motorized rails) were abandoned for a simplified fixed-solenoid approach. Upgrading to larger actuators provided the necessary force to press all four strings simultaneously, though their larger physical footprint restricts the available fret range.
- **Transition speeds:** To avoid damaging the instrument when establishing contact, approach speeds must remain strictly constrained. This necessary safety measure unavoidably introduces noticeable acoustic gaps during string changes.

- **FUTURE WORK**

- **AI audio feedback:** Integrating a microphone could enable a reinforcement learning agent to dynamically adjust bowing force and speed, allowing for real-time acoustic tuning and musical expressivity (e.g., crescendos).
- **Transition optimization:** Dead time between notes could be minimized by using optimized algorithms when changing the strings.
- **Hardware iteration:** Using a more complex mechanical system or employing a secondary robotic arm for fretting would unlock the full fretboard and enable advanced playing techniques like vibrato.
- **Automated calibration:** Replacing manual string registration with force-sensing or vision-based contact detection would ensure faster, repeatable system startup.

5 - DEMO VIDEOS AND REPOSITORIES

The source code for StradiBot is available on GitHub:

- **Main branch:** <https://github.com/Anna-Masset/stradibot/tree/main>

Demo videos of the system in simulation and on the physical hardware are available at the following link:

- **Demo video:** <https://youtu.be/Xr3WuakZd-c>

APPENDICES

